

Transferring data from ARCHER2

William Lucas

EPCC, The University of Edinburgh

w.lucas@epcc.ed.ac.uk

www.archer2.ac.uk



Reusing this material



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

This means you are free to copy and redistribute the material and adapt and build on the material under the following terms: You must give appropriate credit, provide a link to the license and indicate if changes were made. If you adapt or build on the material you must distribute your work under the same license as the original.

Note that this presentation contains images owned by others. Please seek their permission before reusing these images.

Partners



THE UNIVERSITY
of EDINBURGH



**Hewlett Packard
Enterprise**

Overview



- Where to take data
- Triage
 - Transfer, regenerate, ignore
- Transfer preparation
 - Archive, compress, checksum
- SSH-based tools
 - scp, rsync
- rclone
- Globus Online



ARCHER2 end of service



- ARCHER2 will cease service at 17:00 UTC on 20 November 2026.
- Login node access will shut down, jobs will end
- Beyond this point, the home and work file systems will be

NO LONGER AVAILABLE

- Before this date, you must retrieve and copy elsewhere any data on those file systems that you want to keep.
- Re-enabling access is not possible; anything forgotten will be gone.

This means every user has 44 days from now to save their data.



Data destinations



|epcc|

ARCHER2 file systems



The three file systems you're used to:

File system	User data locations
ARCHER2 home	<code>/home/[project ID]/[group ID]/[user ID]</code>
ARCHER2 work	<code>/work/[project ID]/[group ID]/[user ID]</code>
ARCHER2 solid state scratch	<code>/mnt/lustre/a2fs-nvme/[project ID]/[group ID]/[user ID]</code>

They will not be accessible beyond 17:00 on 20 November.

Further access cannot be negotiated.

ARCHER2 file systems



File system	End of access	User data locations	Notes
RDFAaS	31 Mar 2027	/epsrc and /general	Access for projects with RDFAaS allocations until 31 March 2027.
NSCDS/EPCCFS	at least 31 Mar 2027 and likely to mid-2028	/nscds/[project ID]/[group ID]/[user ID]	Available for projects with national HPC resource allocations beyond ARCHER2 EOS.

If your project extends beyond 20 November using other national HPC resources, the PI can request NSCDS/EPCCFS storage via the ARCHER2 Service Desk at support@archer2.ac.uk.

Determine destinations



- Outside of the RDFaaS and NSCDS/EPCCFS, your storage will likely be institutional or at other HPC sites.
- Potentially cloud.
- Maybe your own laptop/desktop machine.
- Most important: how much space/quota do you have on the destination storage?
- Confirm this before starting any transfers, and include in your data transfer plan. There's no sense in starting a transfer of 2 TB of data to a location with only 1 TB available.
 - Keep track of usage with tools like `df` and `du`.



Triaging data



|epcc|

Data triage



- Before you start moving data, start to come up with a plan by asking yourself: “Which data do I *need* to move?”
- Every item you don’t have to transfer makes the process easier.
- ‘Not all data is created equal’:

Category	Example
Irrelevant or obsolete data	Old output, debug information
Cheap to regenerate	Postprocessed data
Expensive to regenerate	Postprocessed data
Irreplaceable data	Original run output, published results

Cheap to regenerate data



- Fairly obvious that expensive to regenerate and irreplaceable data should be transferred, while irrelevant/obsolete data can be ignored.
- But there are real savings to be had with the ‘cheap to regenerate’ category that might be easily missed.
- Typically (not always) postprocessed results. Ask yourself: would it take longer to transfer the data than to regenerate it? If so, consider leaving it behind.
 - Of course, if you do so, you **must** take the input and the scripts to regenerate with you!

Other things you shouldn't forget



- Job scripts
 - You may not have anything else telling you *exactly* how you ran your simulations
 - ‘Was it version 5.5.2, or 5.6.0? Was that option enabled, or not..?’
- Build instructions or scripts
 - Starting point for building software on your next platform
 - As above, may be only record of what options were used to build software
- Postprocessing scripts
 - **Particularly if you intend to use it to regenerate data you're leaving behind!**
- Source code, including local-only branches of repositories
 - Don't leave behind anything you were in the middle of working on
 - Check branches and git stash for anything otherwise missed.

Getting going now

- The ARCHER2 login nodes can sustain ~ 100 MB/s SSH-based transfers.
- At best, this gives ~ 8.6 TB/day.

But everyone else will be doing the same thing!

- That means, start now!
- At some point *before* 20 November it will likely already be too late for you to transfer everything you want.
- Start early so you can make it in time.



Preparing data for transfer



|epcc|

Preparing data for transfer

- How large are your files?
- How many files are there?
- In how many directories are they?
 - In a shallow structure, or many directories deep?
- This will inform whether to archive your data, whether to compress it, and which transfer tool you use.
 - More on all this later.

Archiving data

- It is much faster to transfer a single archive of many files, rather than transferring the files individually
 - Globus can be an exception, see later
- Every file operation comes with Lustre metadata operation overhead, plus SSH encryption in transfer.
- You can expect 500,000 files of 2 kB each to transfer orders of magnitude more slowly than a single 1 GB file.
- You should also consider whether any archive should be compressed or not.
 - More often than not, the answer is no.

Using `tar` effectively



- Archive a campaign directory with:

```
tar -b 2048 -cvWlf campaign_01.tar campaign_01/
```

- `-b 2048` 1 Mb block size to lower metadata contention with other users

- — **Note here: Archive a directory, not files in the**
- — **current directory. We do not want to tarbomb**
- — **ourselves on the other side of the transfer!**

- You could also use `zip`:

```
zip -0r campaign_01.zip campaign_01/
```

- The `-0` option means this is an uncompressed archive.
- However, `zip` doesn't preserve hard links and can't be stream unarchived.

To compress, or not to compress

- Compression makes the archive smaller, though likely not by much, and will take perhaps considerable time to do so.
- With tar, add a `-z` flag for a gzipped `.tar.gz`:
`tar -b 2048 -czvWlf campaign_01.tar.gz campaign_01/`
- Therefore the question is: is more time saved transferring a smaller archive than would be required to compress it in the first place?
- Compression works best:
 - If it is a lot of ASCII text
 - If it is sparse data stored in a dense format
 - If the destination is severely bandwidth limited (e.g. your own home)
- Otherwise, it's very likely faster to create and transfer uncompressed.

Other `tar` tips

- Create an index file you transfer with the `.tar`

- Contains a list of everything in the archive

```
tar -tvf campaign_01.tar > campaign_01.tar.list
```

- (Re)-verify data against the original:

```
tar -df campaign_01.tar campaign_01/
```

- Aim for the size of each `.tar` to be a few GB up to 1 or 2 TB *at most*.
 - Anything bigger than this means something going wrong / any methodology changes could set you back hours.
 - Also allows you to easily fit archive/unarchive/checksum work into jobs that the QoS will accept.
 - If doing so in jobs on the serial partition+QoS – recommended!

Checksums



- It's extremely wise to calculate checksums on both sides (ARCHER2 and destination) to ensure that data transfers correctly.
 - A little boring, perhaps, but potentially a lifesaver.
 - You don't want to go to a .tar for the first time next year and find it corrupted.
- On ARCHER2, create a manifest file containing the checksums of all .tar files matching `campaign_*.tar` in the current directory with
`md5sum campaign_*.tar > manifest.md5`
- Copy the manifest to the destination with everything else.
- Then check that all files match with
`md5sum -c manifest.md5`
- MD5 is probably your best choice for hash function on ARCHER2.

If you want to use SHA256...



- Sha256sum is not installed on ARCHER2; instead, do:

```
openssl dgst -r -sha256 campaign_*.tar \  
> manifest.sha256
```
- Then at destination check that all files in the manifest match with

```
sha256sum -c manifest.sha256
```
- Or calculate a new check file containing all checksums:

```
openssl dgst -r -sha256 campaign_*.tar \  
> check.sha256
```

Then ensure the manifest and check files match:

```
diff <(sort manifest.sha256) \  
<(sort check.sha256) && echo "all match"
```

Build a real plan



- This can be as simple or as complex as you like, but it should exist.
- At an absolute minimum have a list of the items to be transferred which you can tick off as they complete.

ARCHER2 source directory	Archived?	Pre-transfer checksum	Destination	Post-transfer checksum	Unarchived?
/work/t01/t01/auser/dir01	dir01.tar	3613f97d712ec185245d6b07f38	/data/archer2/dir01.tar	3613f97d712ec185245d6b07f38	/data/archer2/dir01
/work/t01/t01/auser/dir02	dir02.tar	f994812238e9d6145994fd4dd4	In progress	-	-
/work/t01/t01/auser/dir03	dir03.tar	In progress	-	-	-

Demo – preparing data for transfer



Transfer tools 1: SSH-based transfer (scp and rsync)



SSH-based transfer

- The simplest option of all.
- Requires nothing except:
 - Pulling data: The ability to log in to ARCHER2 from the destination
 - Alternatively, pushing: the ability to log into the destination from ARCHER2
 - sshd set up and running, IP, port forwarding if pushing to storage at home...
- Otherwise, there is no setup. If your data is archived, you can start moving it right now.
- But, not the fastest – connection is encrypted, which slows it down.
- Best used for archives of a few hundred GB at very most.
 - 1 TB or more could get very difficult

Quick note on usage

- Both of these tools work similarly to `cp`:
`cp [options] /path/to/source /path/to/destination`
- These could be files or directories.
- For us here, one item will be local, e.g. a local directory where we're intending to download our ARCHER2 data:
`/data/archer2`
- The other will be a remote location, which needs to be prepended with the `user@address` you use to log in (or an SSH config `Host` if you have one set up). Here, e.g. a `.tar` in our ARCHER2 work directory:
`auser@login.archer2.ac.uk:/work/t01/t01/auser/data01.tar`

scp



- The simplest tool of all.
- Logged in on the destination machine, pulling from ARCHER2:

```
scp -c aes128-ctr \  
  auser@login.archer2.ac.uk:/path/to/campaign_01.tar \  
  /data/archer2/campaign_01.tar
```

 - We speed this up using `-c aes128-ctr` which sets a lower level of encryption which is hardware accelerated (less stress on CPU) – but still an overhead.
- Or, logged in on ARCHER2, pushing to the destination machine:

```
scp -c aes128-ctr campaign_01.tar \  
  auser@remote.ac.uk:/data/archer2/
```
- When scp completes, `campaign_01.tar` is in `/data/archer2` on the destination machine.

rsync



- Similar to scp but more powerful. Use this instead if you can.
- To pull data from ARCHER2 to local directory /data/archer2:

```
rsync -avP -e "ssh -c aes128-ctr" \  
  auser@login.archer2.ac.uk:/path/to/campaign_01.tar \  
  /data/archer2/campaign_01.tar
```

 - `-a` Archive mode preserves permissions, times, symbolic links...
 - `-v` Verbose to print out what's happening
 - `-P` Enables both `--progress` and `--partial`, allowing transfers to be resumed if interrupted.
 - As with scp we use faster aes128-ctr encryption.
- Reverse source and destination if pushing from ARCHER2 to elsewhere.

rsync tips

- Likely won't saturate with just one transfer (unless restricted by destination bandwidth). Run several transfers at the same time for better throughput.
- Transferring a directory and its contents (rather than an archive) would look like the following:

```
rsync -av --info=progress2 \  
-e "ssh -c aes128-ctr" \  
auser@login.archer2.ac.uk:/path/to/campaign_01 \  
/data/archer2
```

- You can use a `--dry-run` option to see what would be transferred or changed without actually doing so.

SSH keys



- Easiest: Linux/macOS, use the SSH Agent, once per login, e.g.:
`ssh-add /path/to/private/key/id_rsa`
- Alternatively/additionally: place an SSH host in `~/.ssh/config`:
 - Looks like:

```
Host archer2
  HostName login.archer2.ac.uk
  User auser
  IdentityFile /path/to/private/key/id_rsa
```
 - Set Host, User, IdentityFile
 - From now on, you can use the Host you chose in SSH commands e.g.:
`ssh archer2, scp archer2:/path ..., rsync archer2:/path ...`

SSH keys



- The fallback is to pass the SSH key's location as an optional argument.
- For SSH:
`ssh -i /path/to/id_rsa auser@login.archer2.ac.uk`
- This translates to the following scp command:
`scp -i /path/to/id_rsa -c aes128-ctr ...`
- And for rsync:
`rsync -e "ssh -i /path/to/id_rsa -c aes128-ctr" ...`

Demo – using rsync to move data



Transfer tools 2: Parallel and cloud transfers with rclone



|epcc|

rclone



- A tool you install and configure yourself.
- Transfers to cloud storage
 - Dropbox
 - Google Drive
 - Microsoft OneDrive
 - Proton Drive
 - *Many* others. See: <https://rclone.org/#providers>
- SFTP transfers e.g. to other HPC sites
- Local to local (such as to NSCDS/EPCCFS, new RDFaaS mount)
- Built-in parallel transfers give considerable speedup over scp/rsync.

Installing rclone

- Simple binary download, unzip, run:

```
$ wget https://downloads.rclone.org/v1.75.1/rclone-  
v1.75.1-linux-amd64.zip  
$ unzip rclone-v1.75.1-linux-amd64.zip  
$ cd rclone-v1.75.1-linux-amd64/
```

- Head to <https://rclone.org/downloads/> for other downloads

- Either run

```
$ ./rclone [arguments]
```

from this directory, use the absolute path to it, or add the unzipped `rclone-v1.75.1-linux-amd64` directory to your `PATH` environment variable to use rclone from elsewhere.

Configuring rclone for cloud storage



- Side-by-side config requires both ARCHER2 and local (e.g. your own laptop) installations of rclone for authorisation via browser for your account on the cloud service.
- Two sets of documentation to follow:
 - https://rclone.org/remote_setup/ for the side-by-side process
 - E.g. <https://rclone.org/dropbox/> for your cloud service.
- Start with ARCHER2 `rclone config` command, create new remote, then move to local machine with e.g.
`rclone authorize "dropbox"`
to authorise in browser and get token in terminal, then copy that token back into the waiting ARCHER2 prompt.

A very small rclone cheat sheet



Command	Action
<code>rclone copy [-P] [remote:]</path/to/source> [remote:]</path/to/destination></code>	Copy source directory and its contents to destination. -P option shows progress.
<code>rclone copyto [-P] [remote:]</path/to/source/file1> [remote:]</path/to/destination/file2></code>	Copy file from source to destination with optional different name. Directories copied as with <code>copy</code> . -P option shows progress.
<code>rclone sync [--dry-run] [remote:]</path/to/source> [remote:]</path/to/destination></code>	Synchronise the destination with the source, so that the destination is the same as the source. This can delete files! Use <code>--dry-run</code> first to check.
<code>rclone lsf [remote:]</path/to/dir></code>	List contents of <code>/path/to/dir</code>
<code>rclone lsd [remote:]</path/to/dir></code>	List directories of <code>/path/to/dir</code>

Similar syntax to `scp/rsync`: `remote:` prepended to a path indicates that the source/destination is elsewhere. Otherwise, it's local.

SFTP and parallel transfers



- If you are transferring data to a site with SFTP, such as another HPC site, you can get parallel transfers using rclone.
- Set up a new SFTP remote (<https://rclone.org/sftp/>) with `rclone config` just as with the cloud providers, choose sftp, then provide host (login URL), your username, password/SSH key/etc for authentication.
- You may need some assistance from the remote site to configure rclone, though no more than if you were using SFTP another way.
- For performance, experiment with `--transfers <N>` option. By default it's 4, but you can try 8 or other values and see what works.

Demo – rclone with Dropbox



Transfer tools 3: Globus Online for massive data transfer



|epcc|

Globus Online is different from the others



- Globus Online is a web GUI rather than a command line tool
 - CLI is available; plus Globus Community Toolkit for GridFTP.
- If source and destination have Globus 'endpoints' then the entire transfer can be managed in an unattended fashion via the website.
 - Queue a set of transfers, close your laptop and go home.
- Globus Connect Server and a good network connection at both ends enable extremely fast transfers of very large amounts of data.
 - Highly parallel, highly efficient.
 - Best speeds for millions of files, millions of directories, *without* archiving!
- Best option, if possible, for HPC site-to-site transfer.
 - ARCHER2 has achieved ~2.3 GB/s.

Globus Connect Server / Personal?



- For parallel transfers (best speeds) Globus Connect Server must be installed and configured at both ends.
- ARCHER2 has such an endpoint set up and ready to go.
- Globus Connect Server is an admin level installation.
 - With some luck there'll also be one at your destination.
- An alternative is to use Globus Connect Personal:
 - User level installation, but less powerful: limited parallelism.
 - If a GC Server isn't running at the destination, admin may allow you to install and run GC Personal as a user (but check with them first!)
 - Install Globus Connect Personal on your own machine if you want to use that as a destination.

ARCHER2 with Globus Online



- Sign into <https://app.globus.org>
 - Use Globus Identity (<https://globusid.org/>) or institutional account
- Two panel view in File Manager, each one a source/destination.
- First, search for 'Archer2 file systems'
 - First time: need to sign in through the SAFE for authentication.
 - If you have more than one ARCHER2 account, the SAFE will ask which one you want to use.
- Follow documentation for the other site to access the other endpoint
- Select files / directories, set any options, click the arrow to queue the transfer.
- Up to: 3 active tasks, 100 pending tasks, 100 million files in a task

Demo – Globus Online transfer from ARCHER2 to laptop

To archive, or not to archive

- Best speed overall will be achieved by transferring unarchived data between high parallelism endpoints.
- If transferring to an endpoint with low parallelism (Globus Connect Personal) this will be slower than transferring an archive of the same.
- If you suspect you're in this situation, test: start a transfer of your unarchived directory, give it 10-15 minutes, note, cancel.
- Then do the same with an archive of the directory (or quickly create a file of random data from `/dev/urandom` using `base64 | head`).
- If you have several large (TB+) files, you might also get performance from transferring 4 to 8 together. Again: start, give 10 mins, measure.

Globus checksumming



- By default, Globus will run a checksum to ensure that each file has transferred correctly.
- That means you can normally skip doing this yourself.
- *But* when the individual files are very large (order of TB or more) it is possible for the checksum operation to take > 1 hour and time out, failing the transfer.
- Non-default disable the checksum in the transfer options.
 - And now you should calculate yourself, as before.



Choosing which tool to use



|epcc|

Choosing a tool

- scp/rsync: No setup, transfer of smaller (up to few 10s of GB), use rsync for ability to restart interrupted transfers
- rclone: Some setup required, use for cloud storage access, or take advantage of parallelism for more data (10s to 100s GB) transfer between sites or within one system.
- Globus Online: No setup if destination already has endpoint, just need an account, fastest and most robust for very large quantities of data (100s GB+ to TB).
 - Some setup if you need Globus Connect Personal.

General recommendation

If at all possible, try to use Globus:

- Best speeds, hands-off/unattended.
- If the non-ARCHER2 endpoint uses Globus Connect Personal, transfer large archives for best speeds.
 - And remember to disable checksums + DIY.
- If fully set up with Globus Connect Server, don't archive. Transfer files as-is to take advantage of parallel transfers.

rclone is also very handy for providing cloud storage access and giving good performance with parallel intra-site transfers.



Unattended data transfer



|epcc|

Unattended transfers



It would be nice to queue+start+run a data transfer without working around keeping SSH connections open.

- Globus basically does this for you!
- If using scp/rsync/rcclone to push from ARCHER2, run the transfer from inside a job on the serial partition/QoS.
 - Make use of the data transfer/serial nodes
 - Requirement: you must be able to log in to the other side from ARCHER2. What exactly this means is set by the destination, not ARCHER2.
 - Passwords, SSH keys, key passphrase, MFA...
 - Write a job script for the serial partition+QoS, then `sbatch` it

Example:

```
#!/bin/bash
#SBATCH --job-name=archive_transfer
#SBATCH --time=1:00:0
#SBATCH --ntasks=1
#SBATCH --account=[your project]
#SBATCH --partition=serial
#SBATCH --qos=serial

/path/to/rclone --transfers 8 copy \
  /work/t01/t01/auser/path/to/campaign_01 \
  onedrive:/path/to/onedrive/campaign_01
```

Summary



|epcc|

Summary



- Before you start, try to understand:
 - What your data is
 - What data you will be transferring
 - How big it is
 - Where it's going
- Then determine your strategy:
 - Use Globus if you can, and definitely if there are 100s of GB to TBs of data.
 - Rsync and rclone can be used for smaller amounts of data.
 - If you have many smaller files, bundle them into larger archives first and take checksums (rsync, rclone, Globus Connect Personal)
 - Leave data uncompressed
 - Unless the destination has very low bandwidth, or you know the data compresses well



Any questions?



|epcc|